

Análisis de Calidad del Agua con Machine Learning

Ríos de la India — Implementación con PySpark y Scikit-Learn

Dataset

CPCB India
534 estaciones

Stack ML

Keras · Scikit-Learn

Stack HPC

Apache Spark · HDFS

Modelos

ANN · RF · GB



1 Problema y Dataset

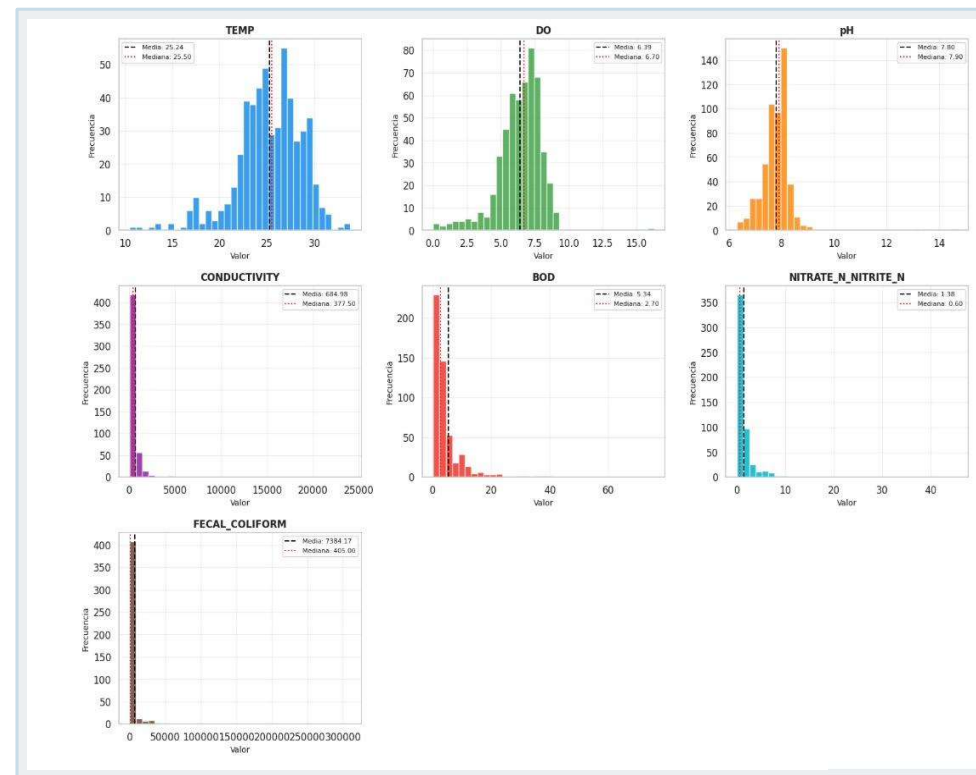
¿Qué queremos predecir?

La calidad del agua en ríos de la India a partir de parámetros fisicoquímicos medidos en campo:

- **Como valor continuo:** Índice WQI (0–100)
- **Como categoría:** Excelente / Buena / Baja / Muy Baja

Variables del dataset

Variable	Unidad	Relevancia
Oxígeno Disuelto	mg/L	★★★★ Alta
pH	—	★★ Media
Conductividad	μS/cm	★★★★ Alta
DBO	mg/L	★★★★ Alta
Nitratos/Nitritos	mg/L	★ Baja
Coliformes Fecales	UFC/100mL	★★★★ Crítica

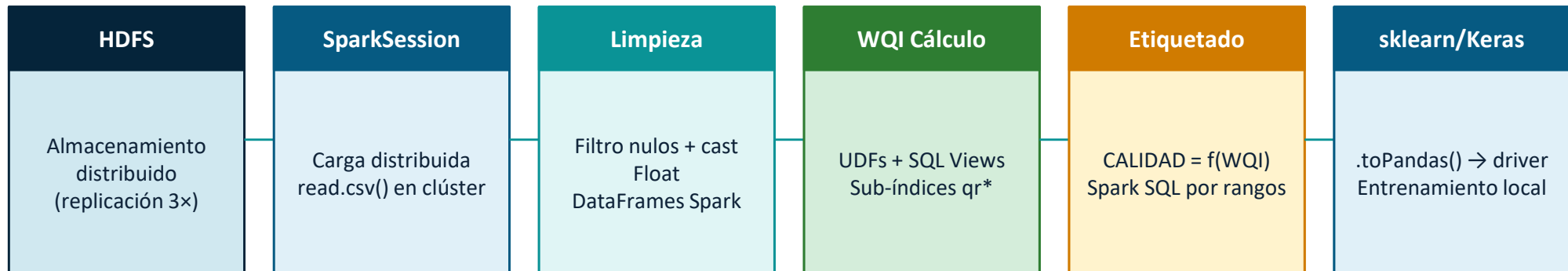


Distribución de parámetros — 534 estaciones CPCB India

534 estaciones · Formato CSV cargado desde HDFS · Procesado con Apache Spark · 4 categorías de calidad



2 Pipeline ETL con Apache Spark + HDFS



¿Por qué Spark? Si el dataset escalara a millones de registros diarios (sensores IoT en tiempo real), el ETL no requiere cambios — solo más nodos en el clúster.

Decisiones de diseño HPC

ETL en Spark, ML en sklearn

Spark no es óptimo para iteración ML — sklearn lo es para ~500 muestras

`n_jobs=-1` en Random Forest

Paralelización perfecta — 300 árboles independientes en todos los núcleos

HDFS como fuente de datos

Desacoplamiento: datos en clúster, cómputo portable

`.toPandas()` solo al final

Minimiza transferencia driver ↔ workers — principio de localidad de datos



3 Índice de Calidad del Agua (WQI)

Cada parámetro se transforma en un sub-índice [0–100] y se pondera según bibliografía científica. Mayor WQI = peor calidad.

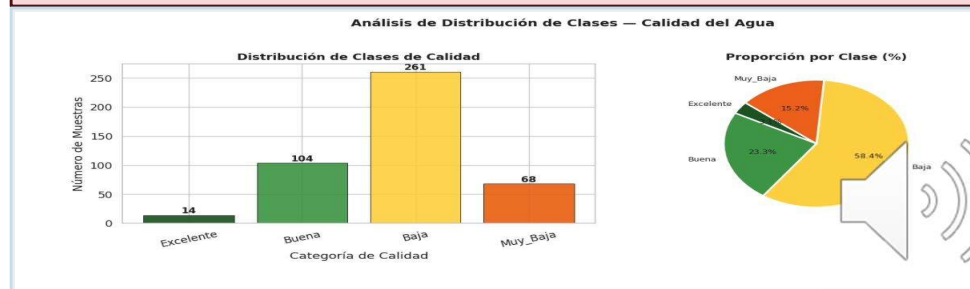
Pesos y sub-índices

Parámetro	Peso	qr = 100 cuando...
Coliformes Fecales	28.4%	< 5 UFC/100mL
Oxígeno Disuelto	28.1%	≥ 6.0 mg/L
Conductividad	23.4%	0–75 µS/cm
pH	16.5%	7.0–8.5
Nitratos/Nitritos	2.8%	< 20 mg/L
DBO	0.9%	< 3.0 mg/L

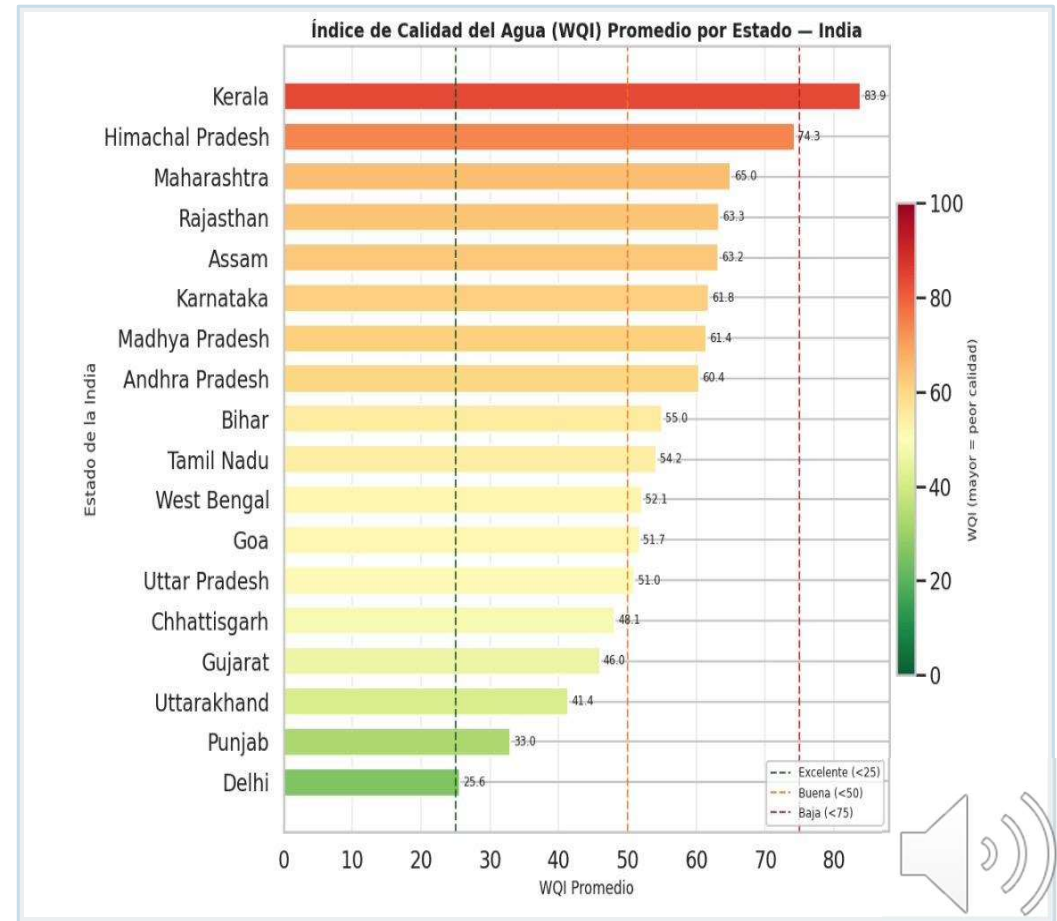
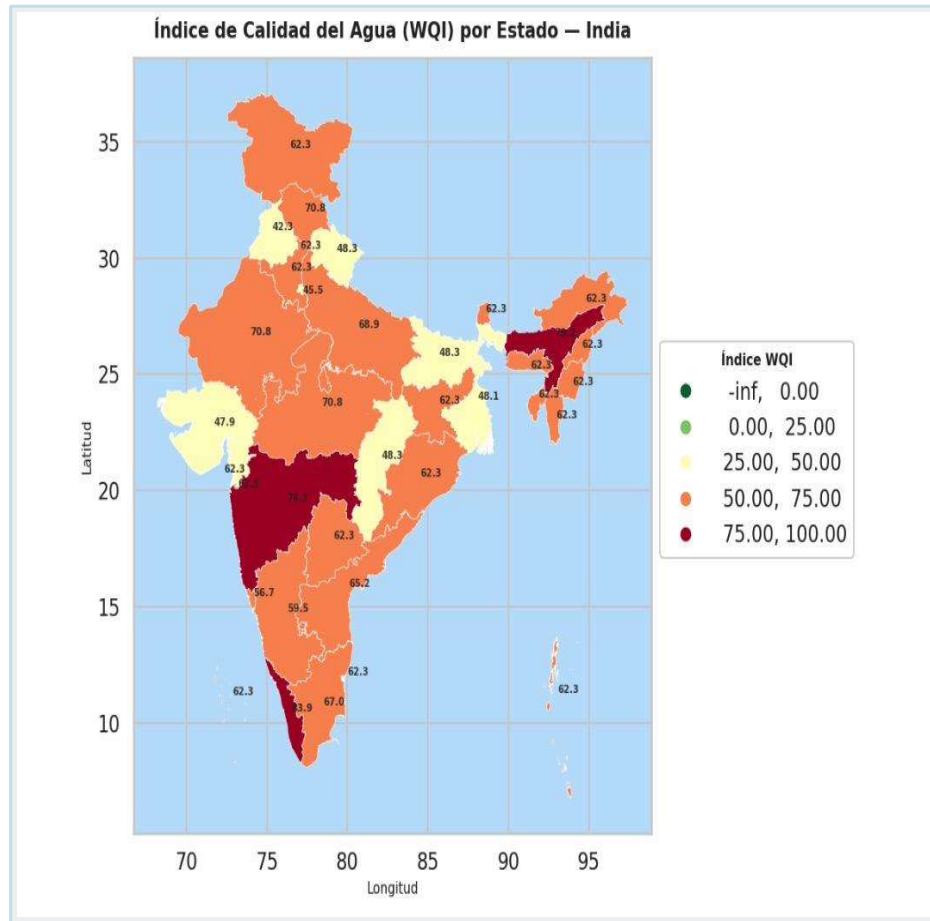
$$WQI = \sum(\text{peso}_i \times \text{qr}_i)$$

Categorías de acción por WQI

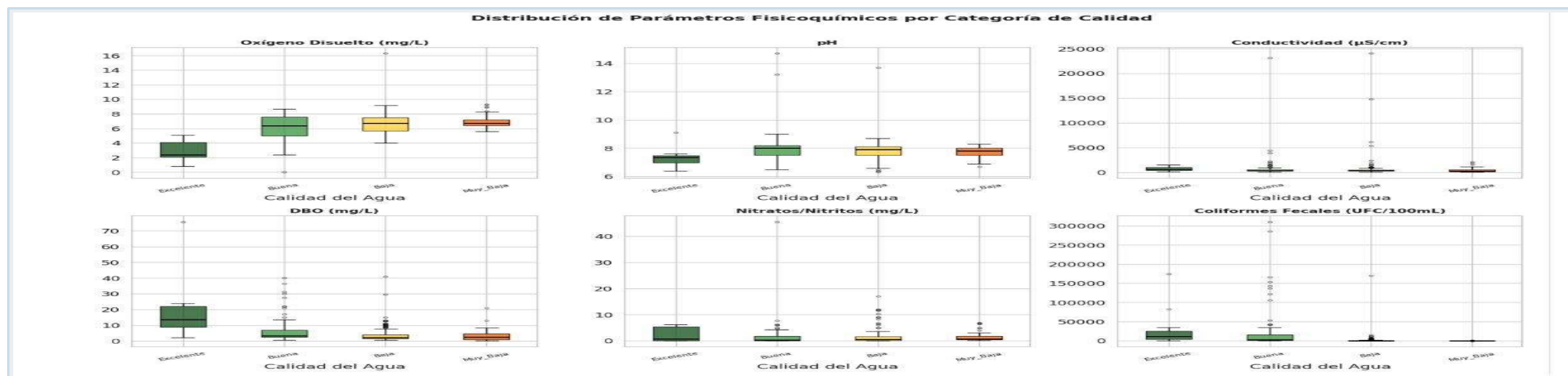
[0 – 25)	Excelente	Apta sin tratamiento
[25 – 50)	Buena	Tratamiento básico
[50 – 75)	Baja	Tratamiento estándar
[75 – 100)	Muy Baja	Tratamiento intensivo
≥ 100	Inadecuada	No apta — alerta crítica



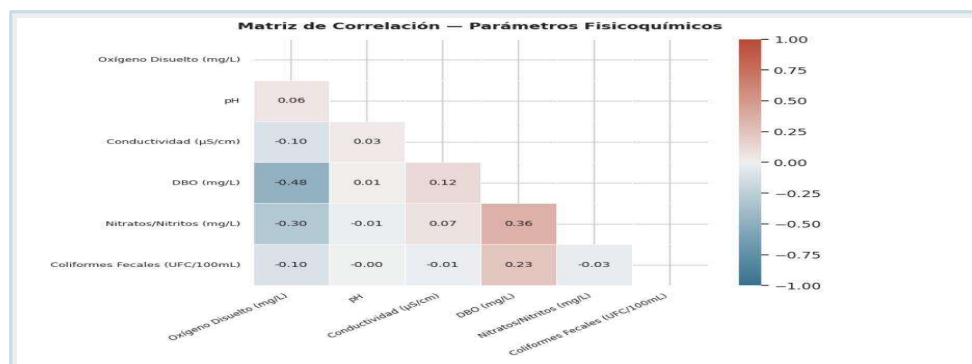
4 Análisis Exploratorio — Distribución Espacial del WQI



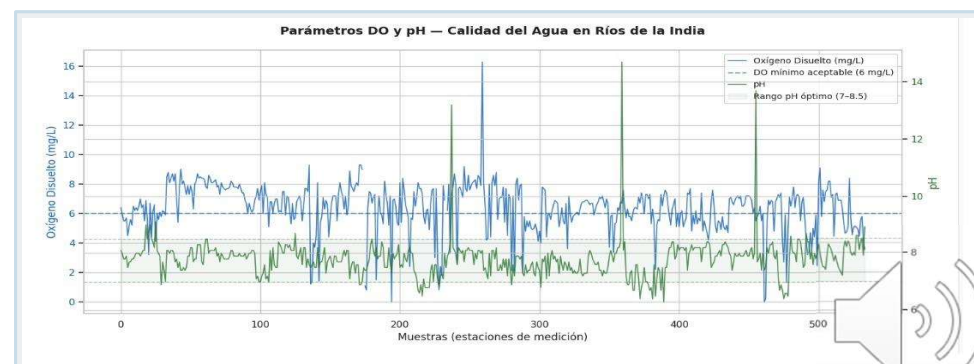
4b Análisis Exploratorio — Parámetros Físicoquímicos



Distribución por categoría de calidad



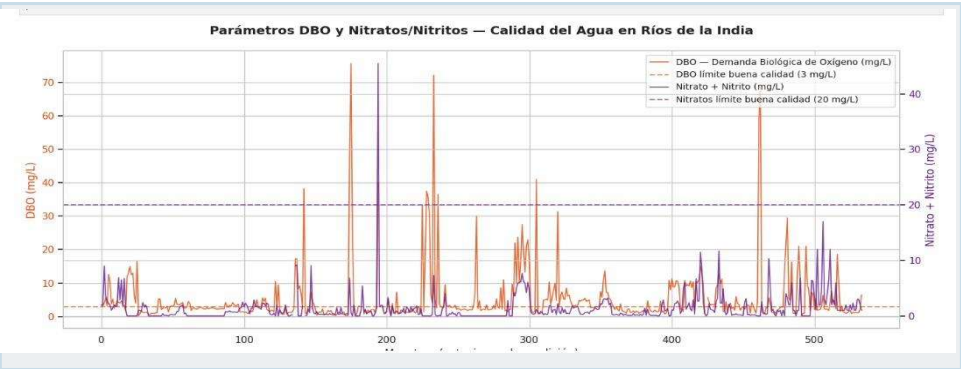
Matriz de correlación



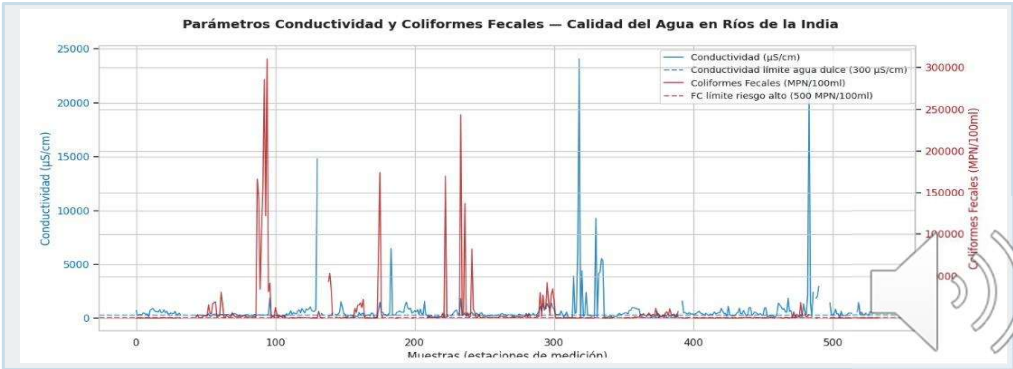
Parámetros DO y pH — series temporales

4c Análisis Exploratorio — Hallazgos Clave

Observación	Implicación para el modelo
DO y BOD son los discriminadores más fuertes	Features con mayor importancia esperada — confirmado por RF y GB
FECAL_COLIFORM con sesgo extremo (~15% outliers)	Árboles de decisión son insensibles a este sesgo — ventaja sobre ANN
DBO ↔ FC: $r > 0.5$ (correlación bioquímica real)	No eliminar — correlación tiene significado real, no es redundancia artificial
Desbalanceo de clases — Baja domina (58%)	Requiere <code>class_weight='balanced'</code> y validación cruzada estratificada
Conductividad con outliers extremos ($>20.000 \mu\text{S/cm}$)	Puntos de contaminación industrial — señal real, no error de medición



DBO y Nitratos/Nitritos

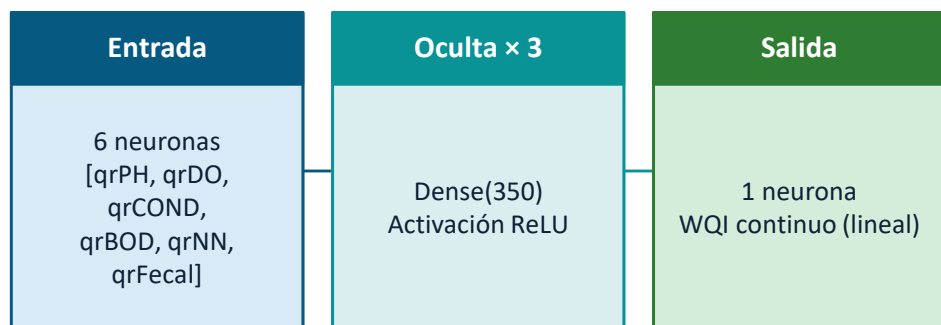


Conductividad y Coliformes Fecales

5 Modelo Base — Red Neuronal (ANN / Keras)

*Regresión: predicción del WQI continuo a partir de sub-índices qr**

Arquitectura



Normalización: StandardScaler · Optimizer: Adam · Loss: MSE · Epochs: 200 · Batch: 16

0.9989

R^2 | 99.89%
varianza explicada

0.51

RMSE | Error
típico ± 0.51 pts
WQI

0.11

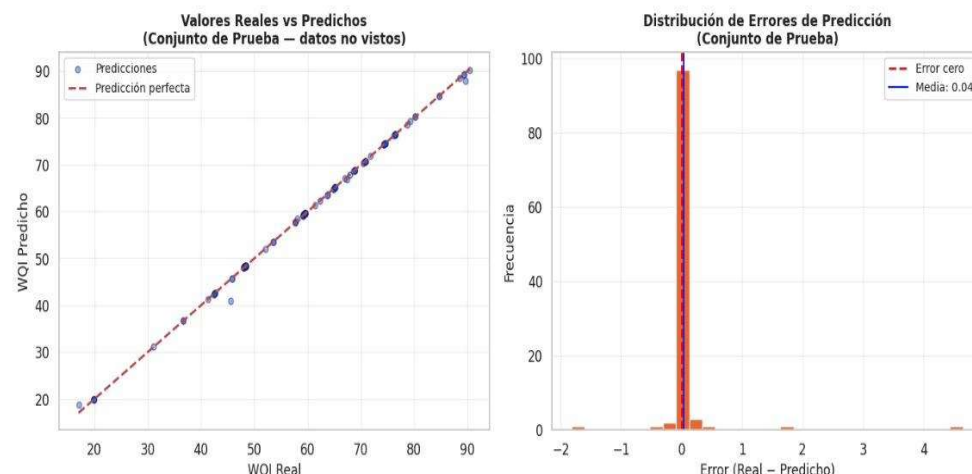
MAE | Error
absoluto
promedio

0.04

Sesgo | Sin sesgo
sistemático

⚠ Limitación: Las features (qr*) son sub-índices calculados con las mismas reglas que producen el WQI. En campo no existen — solo existen las mediciones crudas. Esto motiva los modelos de clasificación directa.

Evaluación del Modelo de Red Neuronal — Conjunto de Prueba (Test Set)



WQI Real vs Predicho + distribución de errores (Test Set)



Curva de aprendizaje (pérdida MSE)

6 Reencuadre: Regresión → Clasificación Directa

¿Por qué agregar nuevos modelos?

Aspecto	ANN — Regresión	RF / GB — Clasificación
Output	Valor numérico WQI	Categoría de acción directa
Features de entrada	Sub-índices derivados (qr^*)	Mediciones crudas directas ✓
Utilidad en campo	Análisis de tendencias	Alertas inmediatas operacionales ✓
Requiere dominio experto	Sí — calcular qr^*	No ✓
Acción del operador	Interpretar un número	Leer etiqueta (Buena/Inadecuada) ✓

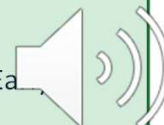
Dos modelos elegidos para clasificación directa

Modelo 1 — Random Forest

Ensemble paralelo · Alta interpretabilidad · Robusto a outliers ·
Entrenamiento paralelizable ($n_jobs=-1$)

Modelo 2 — Gradient Boosting

Ensemble secuencial · Estado del arte en datos tabulares · Ea
stopping automático · Mayor precisión



7 Modelo 1 — Random Forest: Algoritmo

Dos fuentes de aleatoriedad forzada

1

Bootstrap

63% muestras random por árbol (bag)

2

Feature Bagging

√6 features random en cada nodo

3

300 Árboles PARALELOS

Entrenamiento simultáneo ($n_jobs=-1$)

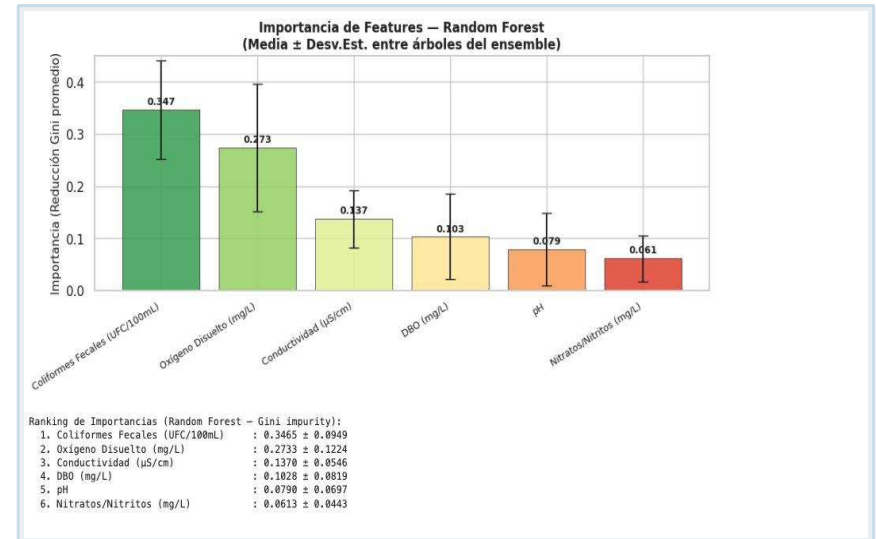
4

Voto por mayoría

Clase más votada = predicción final

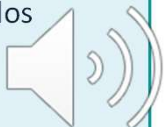
Hiperparámetros

Parámetro	Valor	Justificación
n_estimators	300	300 árboles independientes
max_depth	12	Árboles profundos (alta complejidad individual)
class_weight	balanced	Compensa desbalanceo de clases
n_jobs	-1	Paralelismo total — todos los núcleos disponibles

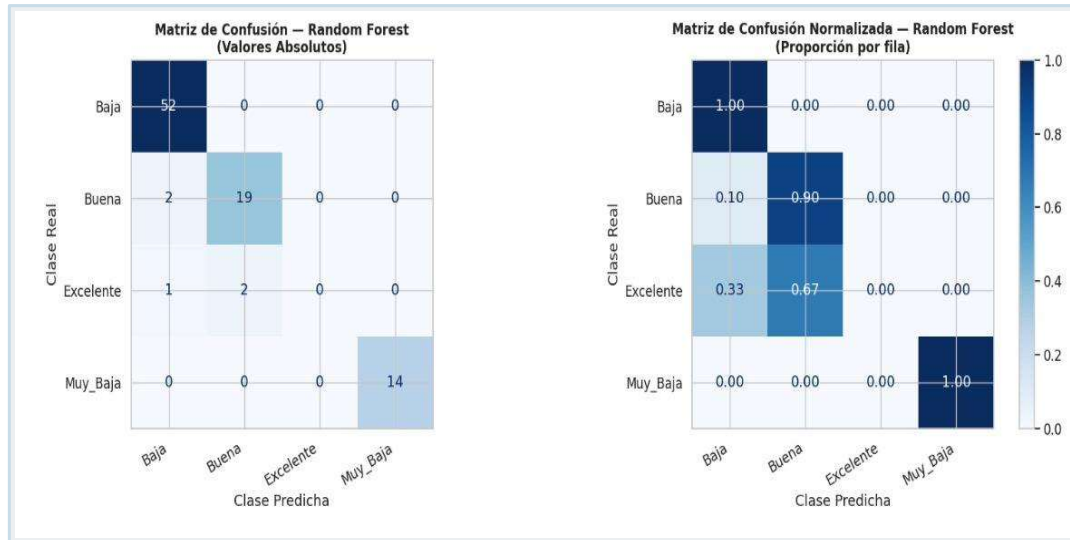


Importancia de Features — Random Forest

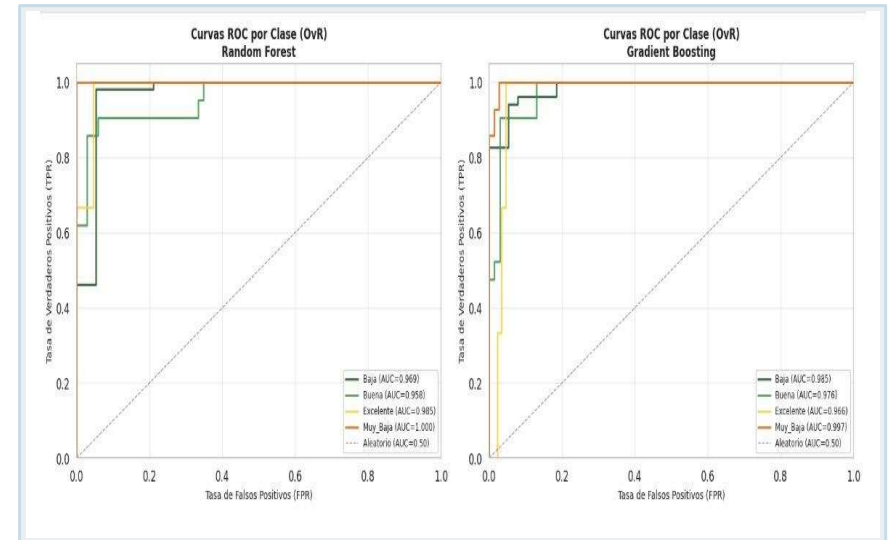
Coliformes Fecales es el predictor más importante (34.7% Gini), seguido por Oxígeno Disuelto (27.3%) y Conductividad (13.7%). Esto valida empíricamente los pesos del WQI original.



7b Random Forest — Resultados



Matriz de Confusión — Random Forest



Curvas ROC por clase (RF y GB)

94.4%

Accuracy

71.3%

Precision
(macro)

72.6%

Recall
(macro)

71.9%

F1-Score
(macro)

97.8%

AUC-ROC
(macro)

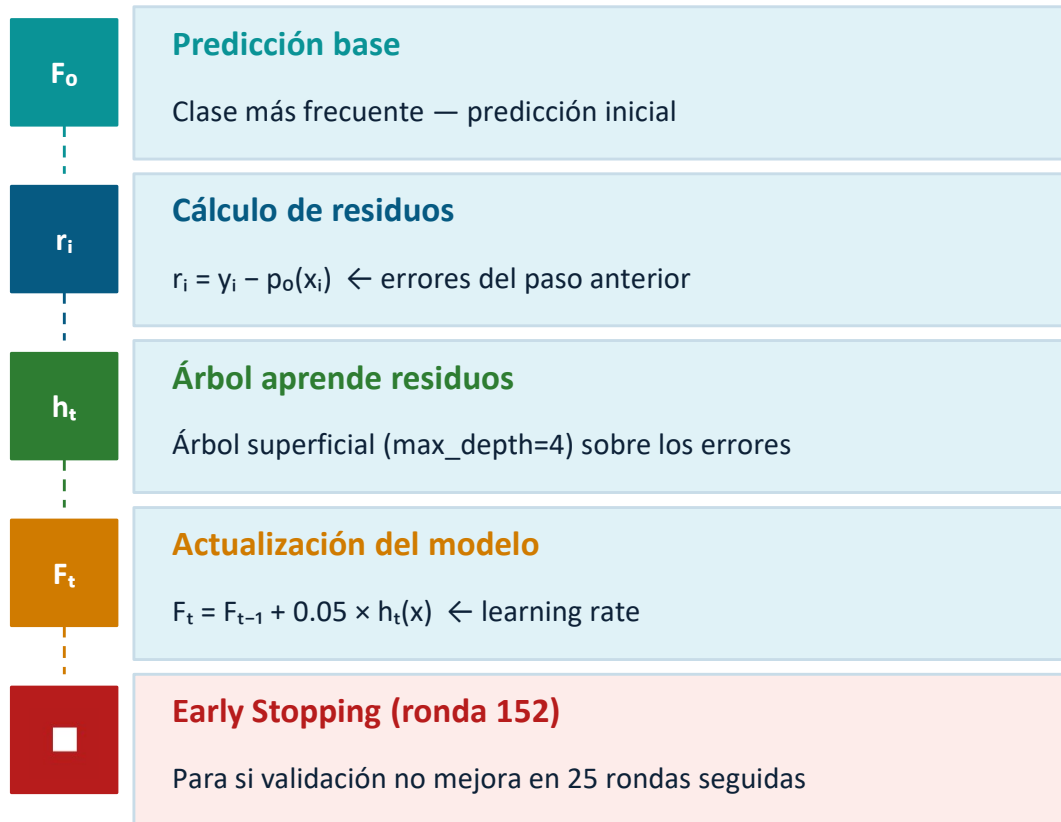
85.1%

CV F1
Me

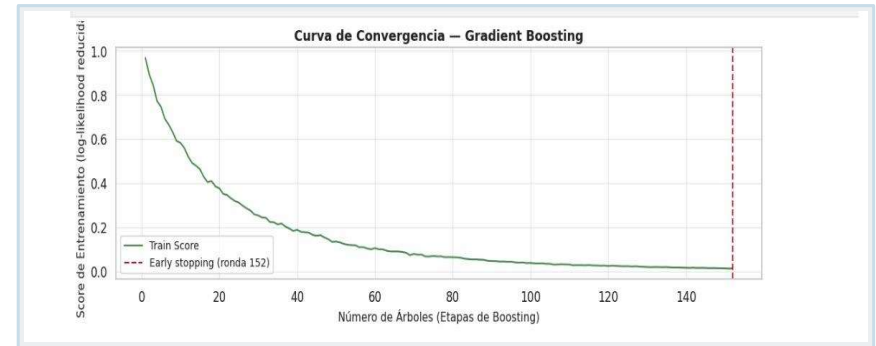
Clase Excelente con menor recall (datos limitados — solo 14 muestras). AUC-ROC Muy_Baja = 1.000 — separación perfecta.

8 Modelo 2 — Gradient Boosting: Algoritmo

Aprendizaje secuencial por corrección de errores



Analogía: aprender a tirar dardos midiendo cuánto fallaste — el siguiente tiro corrige ese error específico.

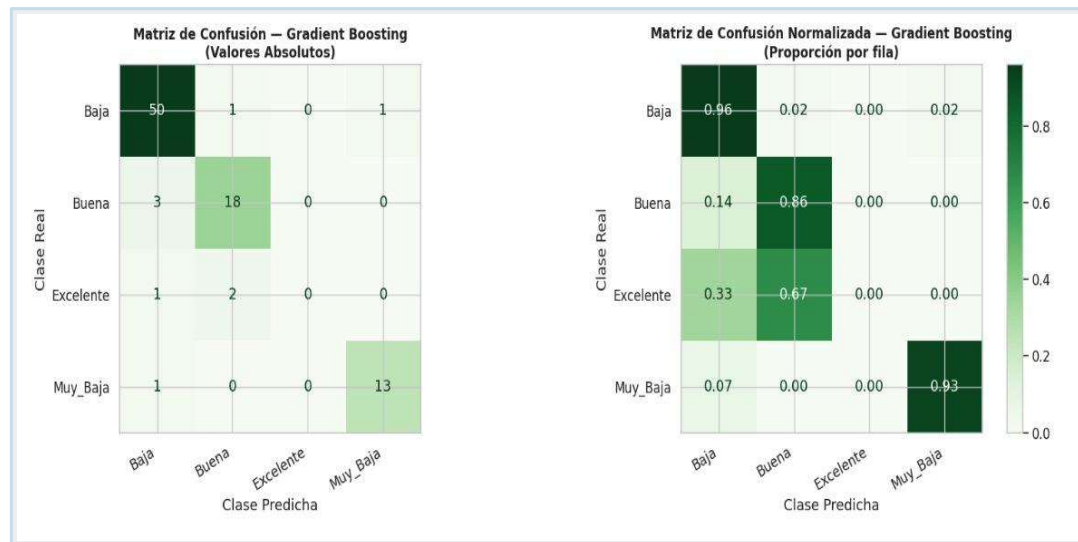


Curva de Convergencia — GB (early stopping ronda 152)

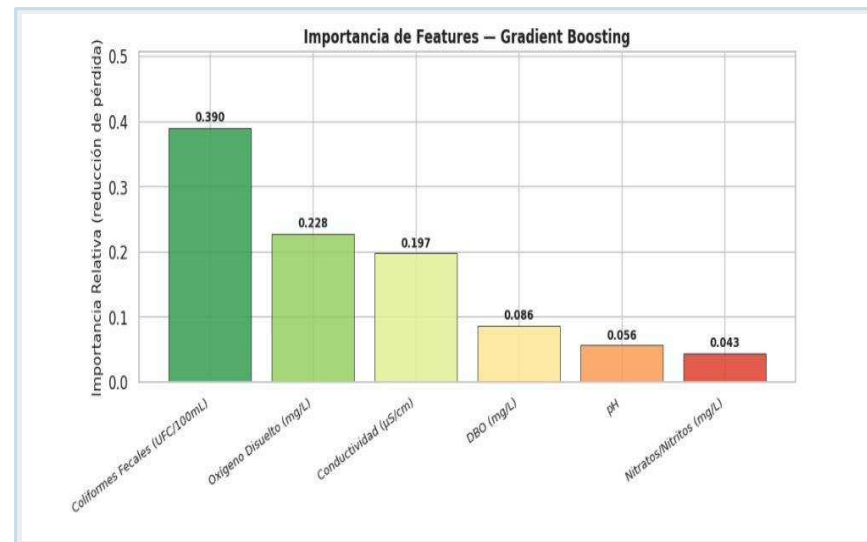
Hiperparámetros clave

Parámetro	Valor	Justificación
n_estimators	300	Límite — early stopping actúa antes
learning_rate	0.05	Ir bajo + muchos árboles = robusto
max_depth	4	Weak learners — evita overfitting
subsample	0.8	Stochastic GB — 20% descartado/árbol
n_iter_no_change	25	Criterio de parada por validación

8b Gradient Boosting — Resultados



Matriz de Confusión — Gradient Boosting



Importancia de Features — Gradient Boosting

90.0%

Accuracy

67.4%

Precision
(macro)

68.7%

Recall
(macro)

68.0%

F1-Score
(macro)

98.1%

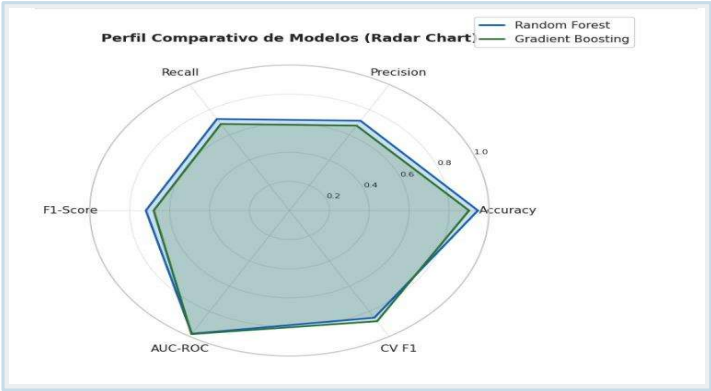
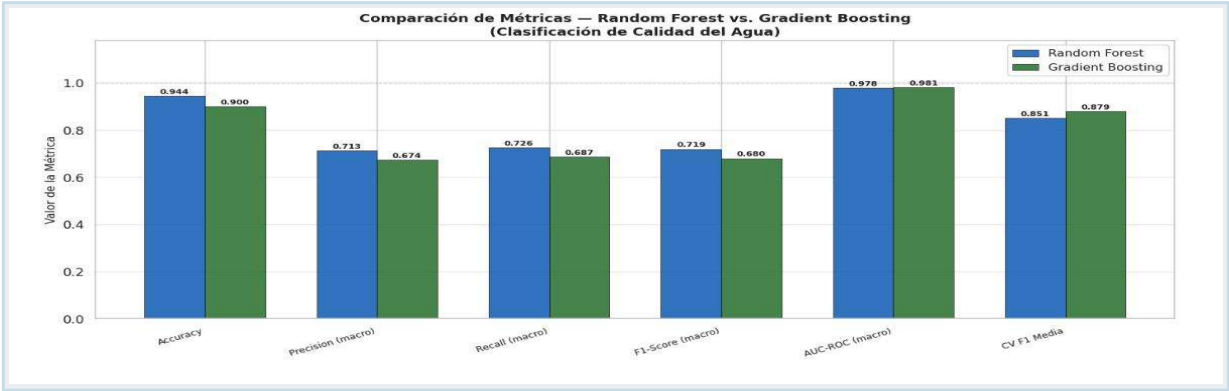
AUC-ROC
(macro)

87.9%

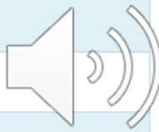
CV F1
Me

GB obtiene mejor CV F1 (87.9% vs 85.1%) y mejor AUC-ROC — recomendado para producción. FC sigue siendo el predictor más importante (39.0%).

9 Random Forest vs. Gradient Boosting — Análisis Comparativo



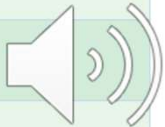
Criterio técnico	Random Forest	Gradient Boosting
Estrategia	Bagging (paralelo)	Boosting (secuencial)
Error que reduce	Varianza	Sesgo
Prof. de árboles	Alta (max_depth=12)	Baja (max_depth=4) — weak learners
Velocidad entreno	Rápida (n_jobs=-1) ✓	Más lenta (secuencial)
Regularización	No automática	Early stopping automático ✓
CV F1 Media	0.851	0.879 ✓
AUC-ROC macro	0.978	0.981 ✓
Uso recomendado	Baseline rápido	Producción ✓



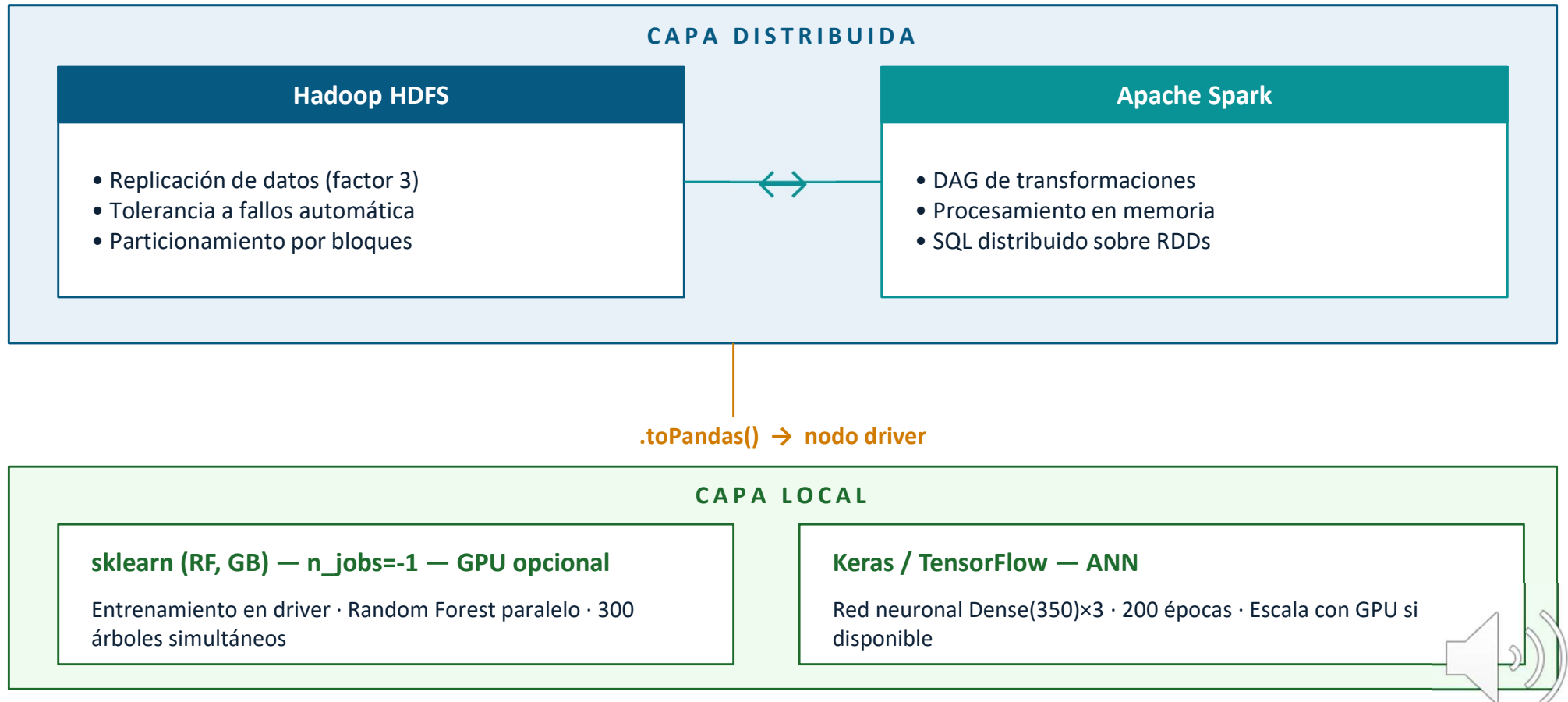
10 Comparación Global — Los Tres Modelos

Los modelos son complementarios — ANN para tendencias continuas del WQI; RF/GB para clasificación operacional directa en campo

Criterio	ANN (Keras)	Random Forest	Gradient Boosting
Tarea	Regresión (WQI)	Clasificación	Clasificación
Features de entrada	Sub-índices qr*	Mediciones crudas ✓	Mediciones crudas ✓
Normalización	Requerida	No necesaria ✓	Buena práctica
Outliers	Sensible	Robusto (árboles) ✓	Robusto (árboles) ✓
Clases desbalanceadas	Manual	class_weight ✓	class_weight + subsampling ✓
Interpretabilidad	Baja (caja negra)	Alta (feature importances) ✓	Media
Overfitting	Riesgo medio	Controlado (bagging)	Early stopping ✓
Escalabilidad	Media	Alta (n_jobs=-1) ✓	Media (secuencial)
Uso recomendado	Tendencias WQI	Baseline rápido	Producción ✓
Mejor métrica	R ² =0.9989 (regresión)	Accuracy 94.4%	CV F1=0.879 / AUC=0.981



11 Arquitectura de Alto Desempeño — Spark + HDFS



12 Conclusiones

Sobre los modelos de Machine Learning

Gradient Boosting: mejor desempeño de clasificación (CV F1=0.879, AUC=0.981) — recomendado para alertas operacionales

Random Forest: alternativa óptima cuando se prioriza velocidad, interpretabilidad y simplicidad de despliegue

ANN complementa con predicción continua del WQI ($R^2=0.9989$) para análisis de tendencias temporales

Los tres modelos coinciden: FECAL_COLIFORM, DO y DBO son las variables más informativas — validación empírica de los pesos del WQI

Sobre la arquitectura de Alto Desempeño

Apache Spark + HDFS: el ETL escala a millones de registros sin modificaciones — condición fundamental del HPC

Separación Spark (ETL) / sklearn (ML iterativo) refleja usar la herramienta correcta en cada capa del pipeline

Paralelización nativa de RF ($n_jobs=-1$): los algoritmos ensemble son inherentemente paralelizables — candidatos ideales en HPC

Validación cruzada estratificada: costo computacional adicional justificado por confiabilidad del resultado en datasets pequeños

Recomendación: Gradient Boosting para producción · Spark+HDFS para ingestión · sklearn para entrenamiento eficiente · ANN para análisis de tendencias continuas



21 Referencias

Dataset y Metodología WQI

- Central Pollution Control Board — CPCB. (2022). National Water Quality Monitoring Programme. Ministry of Environment, India. <https://cpcb.nic.in/>
- Brown, R. M., McClelland, N. I., Deininger, R. A., & Tozer, R. G. (1970). A water quality index: Do we dare? *Water and Sewage Works*, 117(10), 339–343.
- Tyagi, S., Sharma, B., Singh, P., & Dobhal, R. (2013). Water quality assessment in terms of water quality index. *American Journal of Water Resources*, 1(3), 34–38. <https://doi.org/10.12691/ajwr-1-3-3>
- Ingole, N. W. & Bhole, A. G. (2019). Water quality parameters — A review. *InTechOpen*. <https://www.intechopen.com/chapters/69568>

Algoritmos de Machine Learning

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Friedman, J. H. (2002). Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4), 367–378. [https://doi.org/10.1016/S0167-9473\(01\)00065-2](https://doi.org/10.1016/S0167-9473(01)00065-2)
- Fernández-Delgado, M., Cernadas, E., Barro, S., & Amorim, D. (2014). Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15(1), 3133–3181.

Frameworks y Herramientas

- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... & Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation*, 265–283.
- Chollet, F. (2015). Keras [Software]. GitHub. <https://github.com/keras-team/keras>
- Jordahl, K., Van den Bossche, J., Fleischmann, M., Wasserman, J., McBride, J., & Gerard, J. (2020). *geopandas/geopandas: v0.8.1* [Software]. Zenodo. <https://doi.org/10.5281/zenodo.3946761>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51–56.



Resumen Ejecutivo

Problema	Clasificación calidad del agua (4 clases) + regresión WQI continuo
Dataset	534 estaciones · India · 6 parámetros fisicoquímicos (CPCB)
HPC Stack	Apache Spark + Hadoop HDFS — ETL distribuido sin modificar al escalar
ML Stack	Keras/TensorFlow (ANN) + Scikit-Learn (RF, GB) — entrenamiento en driver
Modelo 1 — ANN	Regresión WQI · $R^2=0.9989$ · RMSE=0.51 · MAE=0.11
Modelo 2 — RF	Clasificación · Accuracy 94.4% · AUC-ROC 0.978 · Alta interpretabilidad
Modelo 3 — GB	Clasificación · CV F1=0.879 · AUC-ROC 0.981 · Early stopping ronda 152
Recomendación	Gradient Boosting para producción · Spark+HDFS para ingestión a escala

Juan Camilo Torres Peña · Maestría en Computación de Alto Rendimiento · 2026

Dataset: CPCB India | Stack: PySpark · Hadoop HDFS · Keras · Scikit-Learn · GeoPandas

13 Trabajo Futuro y Mejoras

Mejora	Técnica propuesta	Impacto esperado
Optimización de hiperparámetros	Optuna / BayesSearchCV distribuido con Spark	+2-5% F1
Explicabilidad avanzada	SHAP values — análisis local y global	Confianza regulatoria
Dimensión temporal	Series de tiempo + LSTM por estación	Predicción proactiva
Datos geoespaciales	Encoding distancia a ciudades/industrias	Patrones regionales
Ensemble final	Stacking (RF+GB+ANN) sobre el driver	Máximo desempeño
Despliegue en tiempo real	FastAPI + Docker + Kafka + Spark Streaming	Alertas IoT en ms

Salto natural: Migrar el entrenamiento a Spark MLlib — Random Forest y Gradient Boosted Trees están implementados de forma nativa con paralelización distribuida, eliminando el cuello de botella del .toPandas() al driver.